



DATABASE SYSTEMS

Chap 6 : Transaction & Concurrency control



Learning Objectives

- Develop a comprehensive understanding of database transactions, including their properties and their role in ensuring data integrity during multi-step operations.
- Learn various concurrency control techniques, such as locking, timestamp ordering, and optimistic concurrency control, to handle simultaneous transaction execution while maintaining data consistency.



6.1 Transaction Concept

A **transaction** is a unit of program execution that accesses and possibly updates various data items(made by set of sql command).

E.g., transaction to transfer \$50 from account A to account B:

1. read(A)
2. $A := A - 50$
3. write(A)
4. read(B)
5. $B := B + 50$
6. write(B)

Two main issues to deal with while executing a transition :

- Failures of various kinds, such as hardware failures and system crashes, business logic
- Concurrent execution of multiple transactions

6.1.1 Example of a transaction in Sql

E.g. Simplified sales (the customer buys two units of product unit price 45frws)

```
UPDATE PRODUCT
```

```
SET PROD_QOH = PROD_QOH - 2
```

```
WHERE PROD_CODE = '1558-QW1';
```

```
UPDATE CUSTOMER
```

```
SET CUST_BALANCE = CUST_BALANCE - 90
```

```
WHERE CUST_NUMBER = '10011' ;
```

```
COMMIT;
```

6.1. 2 ACID Properties of transaction

To preserve the integrity of data, the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

6.1. 2 Transaction properties

- Transaction to transfer \$50 from account A to account B:
 1. read(A)
 2. $A := A - 50$
 3. write(A)
 4. read(B)
 5. $B := B + 50$
 6. write(B)
 7. commit
- Atomicity requirement
 - If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
 - Failure could be due to software or hardware
 - The system should ensure that updates of a partially executed transaction are not reflected in the database
- Durability requirement
 - once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database

6.1.2 Transaction properties(Cont.)

- Consistency requirement in above example:
 - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
 - Implicit integrity constraints
 - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction must see a consistent database.
 - During transaction execution the database may be temporarily inconsistent.
 - When the transaction completes successfully the database must be consistent
 - Erroneous transaction logic can lead to inconsistency

6.1.2 Transaction properties((Cont.)

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

T2

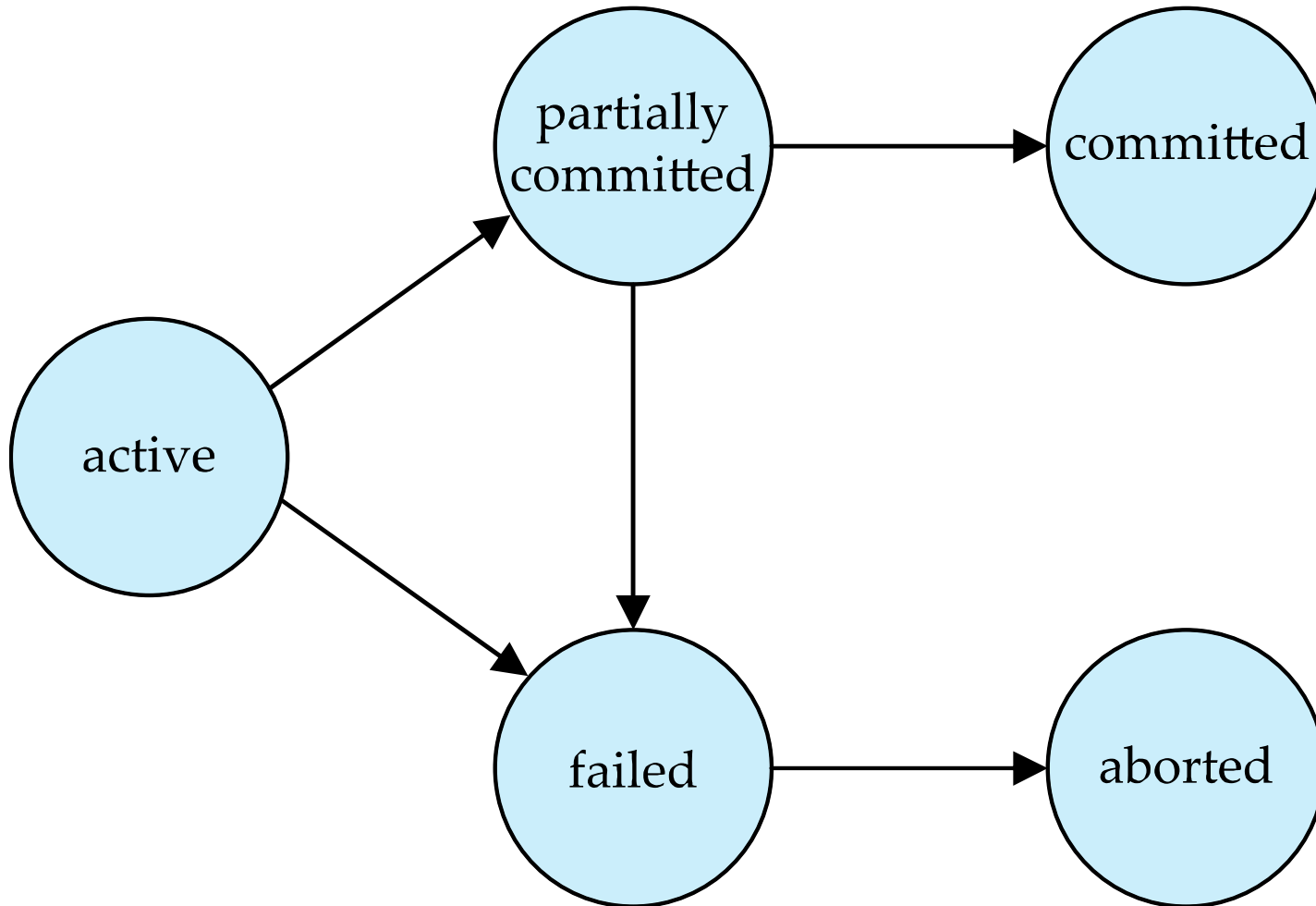
read(A), read(B), print(A+B)

- Isolation can be ensured trivially by running transactions **serially**
- That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

6.1. 3 Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - Restart the transaction
 - Can be done only if no internal logical error
 - Kill the transaction
- **Committed** – after successful completion.

6.1.3 Transaction State (Cont.)



Transaction stages to transfer \$100 from Account A to B.

Stages	Description
Active	- The system reads the balance of Account A and Account B. - It deducts \$100 from Account A and adds \$100 to Account B.
Partially Committed	- money deducted from Account A, added to Account B), but it has not yet been confirmed(WRITE TO DATABASE).
Committed:	- The database writes the final balance to Account A and Account B, making the transaction permanent.
Aborted:	- If, for example, the system crashes right after deducting \$100 from Account A but before adding it to Account B

6.1.4 The Transaction Log

- A DBMS uses a transaction log to keep track of all transactions that update the database.
- The DBMS uses the information stored in this log for a recovery requirement triggered by a ROLLBACK statement.
- The transaction log stores the following:
 - A record for the beginning of the transaction.
 - For each transaction component (SQL statement):
 - The type of operation being performed (INSERT, UPDATE, DELETE).
 - The names of the objects affected by the transaction (the name of the table).
 - The “before” and “after” values for the fields being updated.
 - Pointers to the previous and next transaction log entries for the same transaction.
- Use: `show engine innodb STATUS`, SQL command to see (Active transactions , Locks , Deadlocks, Recovery info)

6.2 Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system , Reduced average response time for transactions, and Increased processor and disk utilization, leading to better transaction throughput.
- Concurrent Executions may result to data inconsistency in a database
 - Need Concurrency control schemes – mechanisms to achieve isolation

6.2. Concurrent transaction Issues

- The three main problems during concurrent transaction are :
 - Lost updates,
 - Uncommitted data,
 - Inconsistent retrievals.
- A DBMS feature that coordinates the simultaneous execution of transactions in a multiprocessing database system while preserving data integrity.
- The objective of concurrency control is to ensure the serializability of transactions in a multiuser database environment.

Lost updates Example

LOST UPDATES

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	$\text{PROD_QOH} = 35 + 100$	
4	T2	$\text{PROD_QOH} = 35 - 30$	
5	T1	Write PROD_QOH (lost update)	135
6	T2	Write PROD_QOH	5

Read Uncommitted data

AN UNCOMMITTED DATA PROBLEM

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	$\text{PROD_QOH} = 135 - 30$	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent Retrieval

- An **inconsistent retrieval** occurs when a **query retrieves data** that is in an **inconsistent state** because a transaction modifying the data is not yet complete.

Example: Initial State:

Account A: \$500 and Account B: \$300

- T1: transfer \$200 from A to B
- T2: Retrieval of balance account A or B

- Inconsistent retrieval happen in case T2 execute before the updating account B.

Account balance A = \$ 500 instead of \$300

balance B = \$ 300 instead of \$ 500

6.3 Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed.
 - A schedule for a set of transactions must consist of all instructions of those transactions
 - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
 - By default transaction assumed to execute **commit instruction** as its last step
 - A transaction that fails to successfully complete its execution will have **an abort instruction** as the last statement

6.3 The scheduler

- The scheduler is The DBMS component that establishes the order in which concurrent transaction operations are executed.
- The scheduler interleaves the execution of database operations in a specific sequence to ensure serializability and isolation of transactions.
- Transactions that are not serializable are executed on a first-come, first-served basis by the DBMS.
- The scheduler's main job is to create a serializable schedule of a transaction's operations,

Schedule 1

- Let **T1** transfer \$50 from A to B, and **T2** transfer 10% of the balance from A to B.
- **A serial schedule** : in which **T1** is followed by **T2** :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 2

- A serial schedule : where T2 is followed by T1

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Schedule 3

- **Interleaved Schedule:** where T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is equivalent to Schedule 1

T_1	T_2
read (A) $A := A - 50$ write (A)	
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	
	read (B) $B := B + temp$ write (B) commit

- In Schedules 1, 2 and 3, the sum $A + B$ is preserved.

Schedule 4

- **Interleaved Schedule:** The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
read (A) $A := A - 50$	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	$B := B + temp$ write (B) commit

6.4 Concurrency Control Method

- Pessimistic
 - ✓ Lock method
 - ✓ Time- stamp
- Optimistic

6.4.1 Concurrency Control with Locking Methods

- Locking methods are one of the most common techniques used to schedule conflicting operation in concurrency control
- Since they facilitate the isolation of data items used in concurrently executing transactions.
- Transaction T2 does not have access to a data item that is currently being used by transaction T1.
- A transaction requires a lock prior to data access; the lock is released after the operation's execution to enable other transactions to lock the data item for their own use.

6.4.1 Concurrency Control with Locking Methods

- Most multiuser DBMSs automatically initiate and enforce locking procedures by **lock manager**.
- **lock manager** is A DBMS component that is responsible for assigning and releasing locks.

Lock Granularity

Level	Description
Database	preventing the use of any tables in the database by transaction T2 while transaction T1 is being executed. but it is unsuitable for multiuser DBMSs.
table	preventing access to any row by transaction T2 while transaction T1 is using the table.
page	the DBMS locks an entire diskpage. A diskpage, or page, is the equivalent of a diskblock,
row	DBMS allows concurrent transactions to access different rows of the same table even when the rows are located on the same page.
Field	The field-level lock allows concurrent transactions to access the same row as long as they require the use of different fields (attributes) within that row.

Lock Types

- Binary
- Shared
- Exclusive.

Binary

- **Binary lock** has only two states: **locked (1)** or **unlocked (0)**.
- If an object such as a database, table, page, or row is locked by a transaction, no other transaction can use that object.
- If an object is unlocked, any transaction can lock the object for its use.

AN EXAMPLE OF A BINARY LOCK			
TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Lock PRODUCT	
2	T1	Read PROD_QOH	15
3	T1	$\text{PROD_QOH} = 15 + 10$	
4	T1	Write PROD_QOH	25
5	T1	Unlock PRODUCT	
6	T2	Lock PRODUCT	
7	T2	Read PROD_QOH	23
8	T2	$\text{PROD_QOH} = 23 - 10$	
9	T2	Write PROD_QOH	13
10	T2	Unlock PRODUCT	

Shared

- A lock that is issued when a transaction requests permission to read data from a database and no exclusive locks are held on the data by another transaction.
- A shared lock allows other read only transactions to access the database.
- Shared locks allow multiple transactions to read the resource concurrently but block all write operations.

Exclusive lock

- An **exclusive lock** is issued when a transaction requests permission to update a data item and no locks are held on that data item by any other transaction.
- An **exclusive lock** does not allow other transactions to access the database.
- **Exclusive locks** block all access (read or write) to the resource while the lock is held.

Example: Shared

Step	Transaction	Operation	Lock Type	Behavior
1	T1	Read Balance	Shared (S)	T1 acquires a shared lock.
2	T2	Read Balance	Shared (S)	T2 also acquires a shared lock.
3	T3	Update Balance	Exclusive (X)	T3 must wait until both T1 and T2 release their shared locks.
4	T1 completes		Shared (S) Released	T1 releases its lock.
5	T2 completes		Shared (S) Released	T2 releases its lock, allowing T3 to proceed.
6	T3	Update Balance	Exclusive (X)	T3 acquires an exclusive lock and updates the record.

Example: Exclusive

Step	Transaction	Operation	Lock Type	Behavior
1	T3	Update Balance	Exclusive (X)	T3 acquires an exclusive lock to update the record.
2	T1	Read Balance	Shared (S)	T1 must wait until T3 releases the exclusive lock.
3	T2	Read Balance	Shared (S)	T2 must wait until T3 releases the exclusive lock.
4	T3 completes		Exclusive (X) Released	T3 releases the exclusive lock.
5	T1	Read Balance	Shared (S)	T1 acquires a shared lock to read the record.
6	T3	Update Balance	Exclusive (X)	T3 acquires an exclusive lock and updates the record.

Two-Phase Locking to Ensure Serializability

- Two-phase locking (2PL) defines how transactions acquire and relinquish locks.
 1. A **growing phase**, in which a transaction acquires all required locks without unlocking any data. Once all locks have been acquired, the transaction is in its **locked point**.
 2. A **shrinking phase**, in which a transaction releases all locks and cannot obtain a new lock.
- The two-phase locking protocol rules:
 - ✓ Two transactions cannot hold conflicting locks (e.g., a shared and an exclusive lock) on the same resource simultaneously.
 - ✓ A transaction cannot release a lock before it **has acquired all the locks it needs**.
 - ✓ No data is affected until all locks are obtained— **that is, until the transaction is in its locked point**.

Example1

Resource: Bank account records in the Accounts table.

Transactions:

1.Transaction T1: Updates Account 101 and Account 102.

2.Transaction T2: Reads Account 101.

Step	Transaction	Operation	Lock State	Behavior	Rule Followed
1	T1	Acquire Exclusive Lock on Account 101	Growing Phase: Exclusive (X)	T1 locks Account 101 for updates.	No conflicting locks (Rule 1).
2	T2	Acquire Shared Lock on Account 101	Blocked	T2 must wait because T1 holds an Exclusive lock.	No conflicting locks (Rule 1).
3	T1	Acquire Exclusive Lock on Account 102	Growing Phase: Exclusive (X)	T1 locks Account 102 for updates.	Locked point is not yet reached (Rule 3).
4	T1	Update Account 101 and Account	Locked Point	Updates occur after all locks are acquired.	All locks obtained before modifying data (Rule 3).
5	T1	Release Lock on Account 101	Shrinking Phase: Exclusive Released	T1 begins releasing locks.	No new locks acquired after unlock (Rule 2).
6	T1	Release Lock on Account 102	Shrinking Phase: Exclusive Released	T1 completes the transaction.	Shrinking phase ends.
7	T2	Acquire Shared Lock on Account	Growing Phase: Shared (S)	T2 proceeds to read Account 101.	Transaction proceeds after locks are relea

Example1 : Key Observations

- **Conflicting Locks:**
 - T2 (shared lock) was blocked until T1 (exclusive lock) released the lock on Account 101.
 - This ensures consistency by preventing concurrent conflicting operations.
- **No Unlock Before Lock:**
 - T1 acquired all locks on Account 101 and Account 102 before releasing them, ensuring no data changes occurred during the "growing phase."
- **Locked Point:**
 - T1 only updated the accounts after acquiring all the locks, ensuring that no partial changes occurred due to incomplete lock acquisition.

Deadlocks

- A **deadlock** occurs when two transactions wait indefinitely for each other to unlock data.
 - T1 = access data items X and Y
 - T2 = access data items Y and X

HOW A DEADLOCK CONDITION IS CREATED

TIME	TRANSACTION	REPLY	LOCK STATUS	
			DATA X	DATA Y
0			Unlocked	Unlocked
1	T1:LOCK(X)	OK	Locked	Unlocked
2	T2:LOCK(Y)	OK	Locked	Locked
3	T1:LOCK(Y)	WAIT	Locked	Locked
4	T2:LOCK(X)	WAIT	Locked	Locked
5	T1:LOCK(Y)	WAIT	Locked	Locked
6	T2:LOCK(X)	WAIT	Locked	Locked
7	T1:LOCK(Y)	WAIT	Locked	Locked
8	T2:LOCK(X)	WAIT	Locked	Locked
9	T1:LOCK(Y)	WAIT	Locked	Locked
***	*****	*****	*****	*****



Deadlocks

- **Deadlock detection:** The DBMS periodically tests the database for deadlocks. If a dead lock is found, the “victim” transaction is aborted.
- **Deadlock avoidance:** **The transaction must obtain all of the locks it needs before it can be executed.**

6.4.2 Concurrency Control with Time Stamping

- Is a transaction management, a technique used in scheduling concurrent transactions that assigns a unique time stamp to each transaction.
- Time stamps must have two properties:
 - Uniqueness**: ensures that no equal time stamp values can exist.
 - Monotonicity**: ensures that time stamp values always increase.
- All database operations (read and write) within the same transaction **must have the same time stamp**.

Time Stamping working principals

- **Wait/Die** : A concurrency control scheme in which an older transaction must wait for the younger transaction to complete and release the locks before requesting the locks itself. Otherwise, the newer transaction dies and is rescheduled.
- **Wound/Wait** : A concurrency control scheme in which an older transaction can request the lock, preempt the younger transaction, and reschedule it. Otherwise, the newer transaction waits until the older transaction finishes.
- **Example** :
 - T1 is Younger (larger timestamp): 19562545
 - T2 is Older (smaller timestamp): 11548789

Time Stamping working principals

Example: - Consider T1 has a time stamp of 11548789 .
T2, has a time stamp of 19562545.

WAIT/DIE AND WOUND/WAIT CONCURRENCY CONTROL SCHEMES			
TRANSACTION REQUESTING LOCK	TRANSACTION OWNING LOCK	WAIT/DIE SCHEME	WOUND/WAIT SCHEME
T1 (11548789)	T2 (19562545)	T1 waits until T2 is completed and T2 releases its locks.	- T1 preempts (rolls back) T2. - T2 is rescheduled using the same time stamp.
T2 (19562545)	T1 (11548789)	- T2 dies (rolls back). - T2 is rescheduled using the same time stamp.	T2 waits until T1 is completed and T1 releases its locks.

6.4.3. Concurrency Control with Optimistic Methods

The optimistic approach is based on the assumption that the majority of database operations do not conflict.

- Using an optimistic approach, each transaction moves through two or three phases, referred to as read, validation, and write.
 1. During the read phase, the transaction reads the database, executes the needed computations, and makes the updates to a private copy of the database values.
 2. During the validation phase, the transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database. If the validation test is positive, the transaction goes to the write phase. If the validation test is negative, the transaction is restarted and the changes are discarded.
 3. During the write phase, the changes are permanently applied to the database.

6.5.Transaction Recovery

- Database transaction recovery uses data in the transaction log to recover a database from an **inconsistent state** to a **consistent state**.
- Database **buffers** are temporary storage areas in primary memory used to speed up disk operations.
- When a transaction updates data, it actually updates the copy of the data in the **buffer** because that process is much faster than accessing the physical disk every time.
- Database **checkpoints** are operations in which the DBMS writes all of its updated buffers in **memory (also known as dirty buffers)** to **disk**. A checkpoint operation is also registered in the **transaction log**.

6.4. 3. Transaction Recovery technique

- **Deferred-write** : the transaction operations do not immediately update the physical database. Instead, only the transaction log is updated. The database is physically updated only with data from committed transactions, using information from the transaction log.
- **Write-through (immediate update)** : the database is immediately updated by transaction operations during the transaction's execution, even before the transaction reaches its commit point.

End